

WebSockets in Go

Contents

1. [Why WebSockets](#)
2. [The protocol](#)
3. [Library choices](#)
4. [Server with gorilla/websocket](#)
5. [Reader and writer pump pattern](#)
6. [The hub pattern \(chat room\)](#)
7. [Server with coder/websocket](#)
8. [Client side](#)
9. [Reconnection](#)
10. [Authentication](#)
11. [Subprotocol](#)
12. [Backpressure and slow clients](#)
13. [Scaling across multiple instances](#)
14. [Production timeouts and limits](#)
15. [Message framing patterns](#)
16. [Binary protocols](#)
17. [SSE vs WebSocket: when to pick SSE](#)
18. [Testing WebSocket handlers](#)
19. [Best practices](#)
20. [Common gotchas](#)
21. [Quick reference](#)

Persistent, bidirectional connections over a single TCP socket. Useful for chat, live dashboards, multiplayer games, collaborative editors. Here's the protocol, the Go libraries, the patterns that hold up in production, and the gotchas.

1. Why WebSockets

HTTP is request/response. The client asks, the server answers, the connection often closes.

WebSockets keep one TCP connection open and let both sides send messages at any time.

	HTTP polling	Long polling	SSE	WebSocket
Direction	Client -> server only	Client -> server only	Server -> client	Bidirectional
Connection lifetime	Per request	Per "wait"	Persistent	Persistent

	HTTP polling	Long polling	SSE	WebSocket
Overhead per message	Full HTTP request	Full HTTP per cycle	Single line	Tiny frame header
Browser support	Universal	Universal	Universal except IE	All modern browsers
Use case	Anything	Compatibility hack	One-way streams	Chat, games, live collab

Pick WebSocket when both sides talk and latency matters. SSE is enough when only the server pushes.

2. The protocol

A WebSocket connection starts with an HTTP `Upgrade` :

```
GET /ws HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

The server responds:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

After 101, the same TCP socket carries WebSocket frames in both directions.

Frame types

Opcode	Name	Use
0x1	Text	UTF-8 text
0x2	Binary	Arbitrary bytes
0x0	Continuation	Continued fragment
0x8	Close	Connection close (with optional status code)
0x9	Ping	Heartbeat
0xA	Pong	Heartbeat reply

The library hides framing. You typically write a message and the library splits or batches it as needed.

3. Library choices

Library	Status	Notes
<code>github.com/gorilla/websocket</code>	Most popular, mature	Wider ecosystem, more examples
<code>github.com/coder/websocket</code> (was nhooyr)	Modern, smaller API	Cleaner, context-aware, recommended for new code
<code>golang.org/x/net/websocket</code>	Deprecated	Don't use
<code>nhooyr.io/websocket</code>	Original home	Now lives at <code>github.com/coder/websocket</code>
<code>github.com/lesismal/nbio</code>	Async/epoll-based	Niche; very high connection count

For new code, `coder/websocket` (formerly `nhooyr/websocket`) is the cleanest. For existing code, `gorilla/websocket` is fine and ubiquitous.

4. Server with gorilla/websocket

Basic upgrade

```
import (
    "net/http"
    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    ReadBufferSize: 4096,
    WriteBufferSize: 4096,
    CheckOrigin: func(r *http.Request) bool {
        return r.Header.Get("Origin") == "https://example.com"
    },
}

func handler(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        return // Upgrade already wrote the error
    }
    defer conn.Close()

    for {
        mt, msg, err := conn.ReadMessage()
        if err != nil {
            return
        }
        if err := conn.WriteMessage(mt, msg); err != nil {
            return
        }
    }
}

func main() {
    http.HandleFunc("/ws", handler)
    http.ListenAndServe(":8080", nil)
}
```

`ReadMessage` blocks until a message arrives or the connection errors. `WriteMessage` sends one message.

Origin check

Without a `CheckOrigin`, gorilla rejects cross-origin upgrades by default. For public APIs:

```
CheckOrigin: func(r *http.Request) bool { return true }
```

Set this knowingly

An open `CheckOrigin` lets any website connect on behalf of a logged-in user. For browser clients with cookies, restrict to known origins.

Concurrent writes need a lock

A WebSocket connection has one write path. Two goroutines calling `WriteMessage` simultaneously corrupt the stream.

```
type Client struct {  
    conn *websocket.Conn  
    mu    sync.Mutex  
}  
  
func (c *Client) Send(msg []byte) error {  
    c.mu.Lock()  
    defer c.mu.Unlock()  
    return c.conn.WriteMessage(websocket.TextMessage, msg)  
}
```

A cleaner pattern: one goroutine owns writes and reads from a channel of outgoing messages (see the writer pump pattern below).

5. Reader and writer pump pattern

The standard structure for a goroutine-per-connection server: two goroutines per connection, one reading, one writing.

```

type Client struct {
    conn *websocket.Conn
    send chan []byte
    hub *Hub
}

const (
    writeWait      = 10 * time.Second
    pongWait       = 60 * time.Second
    pingPeriod     = (pongWait * 9) / 10
    maxMessageSize = 1024 * 1024
)

func (c *Client) readPump() {
    defer func() {
        c.hub.unregister <- c
        c.conn.Close()
    }()

    c.conn.SetReadLimit(maxMessageSize)
    c.conn.SetReadDeadline(time.Now().Add(pongWait))
    c.conn.SetPongHandler(func(string) error {
        c.conn.SetReadDeadline(time.Now().Add(pongWait))
        return nil
    })

    for {
        _, msg, err := c.conn.ReadMessage()
        if err != nil {
            return
        }
        c.hub.broadcast <- msg
    }
}

func (c *Client) writePump() {
    ticker := time.NewTicker(pingPeriod)
    defer func() {
        ticker.Stop()
        c.conn.Close()
    }()

    for {
        select {
        case msg, ok := <-c.send:
            c.conn.SetWriteDeadline(time.Now().Add(writeWait))
            if !ok {
                c.conn.WriteMessage(websocket.CloseMessage, []byte{})
                return
            }
            if err := c.conn.WriteMessage(websocket.TextMessage, msg); err != nil {
                return
            }
        case <-ticker.C:
            c.conn.SetWriteDeadline(time.Now().Add(writeWait))

```

```

        if err := c.conn.WriteMessage(websocket.PingMessage, nil); err != nil {
            return
        }
    }
}

```

Key design choices:

- One reader, one writer goroutine per connection. No write contention.
- Send via channel. Anyone wanting to write pushes to `c.send`.
- Ping/pong keepalive. Server sends pings; client's pong resets the read deadline.
- Read deadline is the disconnect detector. If no pong arrives in `pongWait`, the read times out and the goroutine exits.
- Max message size. Cap to prevent memory abuse.

Start the pumps when the connection is accepted:

```

go client.writePump()
go client.readPump()

```

`readPump` blocks until disconnect. `writePump` exits when the channel closes or the ticker fails.

6. The hub pattern (chat room)

For broadcasting messages to many clients:

```

type Hub struct {
    clients    map[*Client]bool
    broadcast  chan []byte
    register   chan *Client
    unregister chan *Client
}

func NewHub() *Hub {
    return &Hub{
        clients:    map[*Client]bool{},
        broadcast:  make(chan []byte, 256),
        register:   make(chan *Client),
        unregister: make(chan *Client),
    }
}

func (h *Hub) Run() {
    for {
        select {
        case c := <-h.register:
            h.clients[c] = true
        case c := <-h.unregister:
            if _, ok := h.clients[c]; ok {
                delete(h.clients, c)
                close(c.send)
            }
        case msg := <-h.broadcast:
            for c := range h.clients {
                select {
                case c.send <- msg:
                default:
                    close(c.send)
                    delete(h.clients, c)
                }
            }
        }
    }
}

```

The hub owns the map of clients. All mutation goes through channels, so there's no mutex. Adding `select { default }` on broadcast drops messages to slow clients instead of blocking everyone.

7. Server with coder/websocket

The newer library has a smaller API and uses `context.Context`.


```

import "github.com/coder/websocket"

func handler(w http.ResponseWriter, r *http.Request) {
    c, err := websocket.Accept(w, r, &websocket.AcceptOptions{
        OriginPatterns: []string{"example.com"},
    })
    if err != nil {
        return
    }
    defer c.CloseNow()

    ctx := r.Context()
    for {
        typ, data, err := c.Read(ctx)
        if err != nil {
            return
        }
        if err := c.Write(ctx, typ, data); err != nil {
            return
        }
    }
}

```

Differences vs gorilla:

- `Accept` instead of `Upgrade`.
- `Read` and `Write` take `ctx`, so cancellation works naturally.
- No global upgrader; options inline.
- Built-in concurrency safety on `Write` (it uses a mutex internally).
- Cleaner Close semantics (`CloseNow` for immediate, `Close(code, reason)` for graceful).

For new services, this API is easier to live with.

8. Client side

gorilla client

```
import "github.com/gorilla/websocket"

dialer := websocket.Dialer{
    HandshakeTimeout: 10 * time.Second,
}
conn, _, err := dialer.Dial("wss://example.com/ws", http.Header{
    "Authorization": []string{"Bearer " + token},
})
if err != nil { return err }
defer conn.Close()

// write
conn.WriteMessage(websocket.TextMessage, []byte("hello"))

// read
_, msg, err := conn.ReadMessage()
```

coder client

```
c, _, err := websocket.Dial(ctx, "wss://example.com/ws", &websocket.DialOptions{
    HTTPHeader: http.Header{"Authorization": []string{"Bearer " + token}},
})
if err != nil { return err }
defer c.CloseNow()

c.Write(ctx, websocket.MessageText, []byte("hello"))
_, msg, err := c.Read(ctx)
```

9. Reconnection

WebSocket clients lose connections. Production clients implement automatic reconnection with backoff.

```

func connectWithRetry(ctx context.Context, url string) {
    backoff := time.Second
    const maxBackoff = time.Minute

    for {
        conn, _, err := dialer.DialContext(ctx, url, nil)
        if err != nil {
            select {
            case <-ctx.Done():
                return
            case <-time.After(backoff):
                backoff = min(backoff*2, maxBackoff)
                continue
            }
        }
        backoff = time.Second // reset on success
        run(conn)             // blocks until disconnect
        // loop and reconnect
    }
}

```

After reconnect, you may need to:

- Re-authenticate.
- Re-subscribe to topics.
- Replay missed messages (requires server-side message buffer or resumable streams).

10. Authentication

WebSocket upgrades go through HTTP first, so any HTTP auth works:

Bearer token in header

```

// client
header := http.Header{"Authorization": []string{"Bearer " + token}}
conn, _, _ := dialer.Dial(url, header)

// server (before Upgrade)
token := r.Header.Get("Authorization")
if !valid(token) {
    http.Error(w, "unauthorized", 401)
    return
}
conn, _ := upgrader.Upgrade(w, r, nil)

```

Cookie

If your site already uses session cookies, the browser sends them automatically on the upgrade.

Token in query string

Some browser libraries can't set custom headers. Token in query is the workaround, with caveats:

- Token shows up in server logs.
- Token may be cached by intermediaries.
- Use short-lived tokens minted just for the connection.

```
token := r.URL.Query().Get("token")
```

11. Subprotocol

`Sec-WebSocket-Protocol` carries a subprotocol identifier. Some libraries (especially in browsers) abuse this header to ship a token because they can't set custom headers.

```
new WebSocket("wss://...", ["bearer.eyJhbGciOiI..."])
```

Server reads it from `r.Header.Get("Sec-WebSocket-Protocol")` and echoes back the chosen subprotocol.

12. Backpressure and slow clients

A slow client whose receive buffer fills up will block your server's write. Without backpressure handling, slow clients can pile up and exhaust memory.

Strategies:

1. Drop messages to slow clients. Use `select default` on the send channel.
2. Disconnect slow clients. Close `send` after one drop; client gets disconnected.
3. Per-client send buffer size. Larger buffer = more tolerance, more memory.
4. Coalesce updates. For state push, send only the latest snapshot.

```
select {
case c.send <- msg:
default:
    close(c.send) // disconnect
    delete(h.clients, c)
}
```

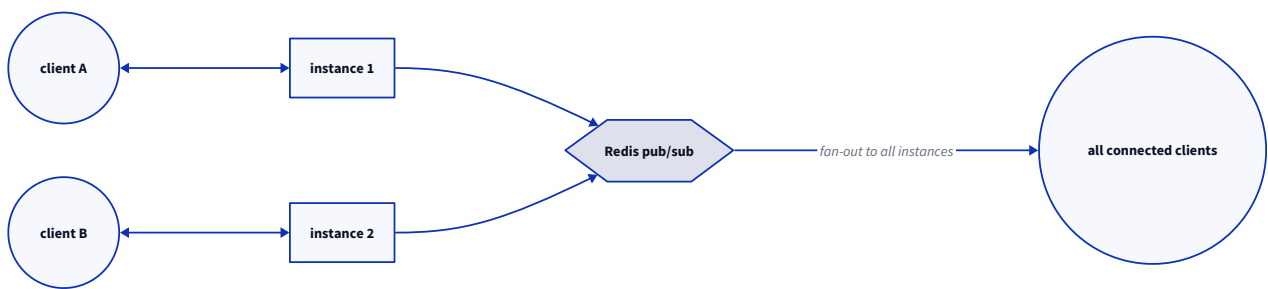
For pub/sub with thousands of clients, dropping is the only sane default.

13. Scaling across multiple instances

A WebSocket connection is stateful. Once a client connects to instance A, future messages from that client go to A, and only A can deliver messages to that client.

For broadcasting across instances:

- 1. Sticky load balancing by client (consistent hashing or sticky session). Each client always lands on the same instance.
- 2. External pub/sub fanout. Any instance publishes to Redis/NATS/Kafka; every instance subscribes and delivers to its local clients.



The sticky requirement is for client to instance affinity. The fanout layer is for message to all instances.

For very high connection counts (>10K per instance), look into async I/O libraries (`lesismal/nbio`) or specialized WebSocket-first frameworks.

14. Production timeouts and limits

Always set:

Setting	Why
<code>SetReadDeadline</code>	Detect dead clients (pong fails)
<code>SetWriteDeadline</code>	Don't hang on slow clients
<code>SetReadLimit</code>	Cap message size; prevent memory abuse
Connection idle timeout	Reclaim memory from abandoned connections
<code>HandshakeTimeout</code> (client)	Don't hang on bad upgrades
<code>ReadBufferSize</code> / <code>WriteBufferSize</code>	Tune for your message sizes

Read deadlines are not optional

Without read deadlines, a disconnected client (e.g., laptop closed mid-WebSocket) holds a goroutine and memory forever until TCP eventually times out (often hours).

15. Message framing patterns

WebSocket messages have a type (text or binary) but no application-level type. For multi-event protocols, add envelopes.

JSON envelope

```
{"type": "chat", "payload": {"user": "alice", "msg": "hi"}}
{"type": "ping", "payload": null}
{"type": "presence", "payload": {"user": "bob", "status": "online"}}
```

```
type Envelope struct {
    Type      string      `json:"type"`
    Payload   json.RawMessage `json:"payload"`
}

var env Envelope
json.Unmarshal(msg, &env)

switch env.Type {
case "chat":
    var chat ChatMsg
    json.Unmarshal(env.Payload, &chat)
    handleChat(chat)
case "ping":
    handlePing()
}
```

`json.RawMessage` defers parsing of the payload until you know the type.

16. Binary protocols

For high-throughput cases (games, market data), use a compact binary format:

- Protobuf
- MessagePack
- FlatBuffers
- Custom byte layout

Set the message type to `BinaryMessage` and write the encoded bytes.

17. SSE vs WebSocket: when to pick SSE

If you only need server -> client streaming and don't need binary, Server-Sent Events (SSE) is simpler:

- Plain HTTP (no Upgrade).

- Goes through more proxies cleanly.
- Browser handles reconnection automatically.
- `text/event-stream` content type.
- No client-to-server channel (use a separate POST endpoint).

```
func sse(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")

    flusher := w.(http.Flusher)
    for {
        select {
        case ev := <-events:
            fmt.Fprintf(w, "data: %s\n\n", ev)
            flusher.Flush()
        case <-r.Context().Done():
            return
        }
    }
}
```

Use SSE when the server pushes and the client only ever needs to ack via a separate endpoint. WebSocket for true bidirectional traffic.

18. Testing WebSocket handlers

`httptest.NewServer` works since the upgrade is HTTP.

```
func TestEcho(t *testing.T) {
    s := httptest.NewServer(http.HandlerFunc(handler))
    defer s.Close()

    wsURL := "ws" + strings.TrimPrefix(s.URL, "http") + "/ws"
    conn, _, err := websocket.DefaultDialer.Dial(wsURL, nil)
    if err != nil { t.Fatal(err) }
    defer conn.Close()

    conn.WriteMessage(websocket.TextMessage, []byte("hello"))

    _, msg, err := conn.ReadMessage()
    if err != nil { t.Fatal(err) }
    if string(msg) != "hello" {
        t.Fatalf("got %q want %q", msg, "hello")
    }
}
```

For unit tests of business logic that uses a connection, mock the read/write interface rather than the whole connection.

19. Best practices

1. Always validate Origin unless your endpoint is truly public.
2. Use one read goroutine and one write goroutine per connection. Multiplex sends through a channel.
3. Cap message size with `SetReadLimit`. Untrusted input can be huge.
4. Set read/write deadlines. Use ping/pong to keep them rolling.
5. Drop or disconnect slow clients. Don't let one slow client back up the broadcaster.
6. Use `wss://` in production. Plain `ws://` over public networks is the equivalent of HTTP.
7. Authenticate at upgrade time, not after. Reject the upgrade if the user isn't allowed.
8. Use JSON envelopes for multi-event protocols.
9. Plan for reconnects. Resubscribe, redeliver, replay as needed.
10. For multi-instance, use Redis/NATS fan-out + sticky sessions.
11. Prefer `coder/websocket` for new code. Context-aware, smaller API.
12. Monitor: open connections, dropped messages, slow client disconnects, ping/pong RTT.

20. Common gotchas

Gotcha	Symptom	Fix
Concurrent writes corrupt the stream	Garbled output, panics	One writer goroutine via channel
No <code>CheckOrigin</code>	Random websites connect on behalf of logged-in users	Whitelist allowed origins
No read deadline	Dead clients hold goroutines forever	Set deadline; use ping/pong to reset
No <code>SetReadLimit</code>	Memory exhaust on huge messages	Cap message size
Closing send channel from broadcaster	Panics on subsequent write	Close via unregister channel, not directly
Forgot to close <code>c.conn</code> on error path	Connection leak	<code>defer conn.Close()</code> at the top
Pinging without pong handler	Read deadline never resets, disconnect	<code>SetPongHandler</code> updates deadline
Browser can't send custom auth header	Connection unauthenticated	Use cookies or subprotocol trick
Plain <code>ws://</code> in production	Tokens travel in cleartext	Use <code>wss://</code>
Single-instance design	Doesn't scale past one server	Add Redis pub/sub + sticky LB
Blocking writes on slow clients	Whole broadcaster stalls	Drop or disconnect on full buffer
Reading the same connection from two goroutines	Race + garbled state	One reader only

Gotcha	Symptom	Fix
Writing JSON with <code>WriteJSON</code> and ignoring errors	Silent disconnect	Check the error
Trusting Origin from non-browser clients	Spoofable; not a security feature	Don't rely on Origin for non-browser auth
Forgetting to set TLS config in WSS	Insecure verify or wrong handshake	Pass <code>TLSClientConfig</code> or rely on system roots
Goroutines leak after disconnect	Counter grows over time	Ensure both pumps exit; use <code>defer</code> to clean up
Reusing Upgrader is fine, but per-connection state isn't	Crosstalk between connections	Each connection gets fresh <code>*Client</code>

21. Quick reference

What	Code
Gorilla upgrade	<code>upgrader.Upgrade(w, r, nil)</code>
Coder accept	<code>websocket.Accept(w, r, opts)</code>
Read message (gorilla)	<code>mt, msg, err := conn.ReadMessage()</code>
Write message (gorilla)	<code>conn.WriteMessage(websocket.TextMessage, b)</code>
Read JSON (gorilla)	<code>conn.ReadJSON(&v)</code>
Write JSON (gorilla)	<code>conn.WriteJSON(v)</code>
Read (coder)	<code>typ, data, err := c.Read(ctx)</code>
Write (coder)	<code>c.Write(ctx, typ, data)</code>
Origin check	<code>Upgrader{CheckOrigin: func(r) bool { ... }}</code>
Max message size	<code>conn.SetReadLimit(N)</code>
Read deadline	<code>conn.SetReadDeadline(time.Now().Add(t))</code>
Write deadline	<code>conn.SetWriteDeadline(time.Now().Add(t))</code>
Pong handler	<code>conn.SetPongHandler(func(string) error { ... })</code>
Send ping	<code>conn.WriteMessage(websocket.PingMessage, nil)</code>
Close with code	<code>conn.WriteMessage(websocket.CloseMessage, websocket.FormatCloseMessage(code, ""))</code>
Reader pump pattern	Single goroutine, blocks in <code>ReadMessage</code>

What	Code
Writer pump pattern	Single goroutine, selects on send channel + ping ticker
Hub pattern	Goroutine with select on register/unregister/broadcast
Auth	Header at upgrade time, or subprotocol, or cookie
Backpressure	<code>select { case send <- msg: default: drop or disconnect }</code>
Multi-instance	Redis pub/sub fan-out + sticky LB
Dialer (gorilla)	<code>websocket.DefaultDialer.Dial(url, header)</code>
Reconnect	Backoff loop wrapping Dial
Testing	<code>httptest.NewServer</code> + <code>ws://</code> URL rewrite
TLS	Use <code>wss://</code> and <code>TLSClientConfig</code>
SSE alternative	One-way push, simpler protocol, plain HTTP